

■ Lab 3 — State Management

■ Objectifs du Lab

À la fin de ce lab, vous aurez :

- Exploré le **fichier d'état Terraform** (`terraform.tfstate`)
- Listé et inspecté les ressources avec `terraform state`
- Renommé une ressource dans le state **sans la recréer**
- Rafraîchi le state avec `terraform refresh`
- Compris la différence entre **desired state** et **current state**

■ Étape 1 — Préparation

Placez-vous dans le dossier du lab :

```
cd labs/lab03-state
```

Créez les fichiers :

```
lab3-state/  
■■■ versions.tf  
■■■ variables.tf  
■■■ main.tf  
■■■ outputs.tf
```

versions.tf

```
terraform {  
  required_version = ">= 1.15.0"  
  
  required_providers {  
    aws = {  
      source = "hashicorp/aws"  
      version = "~> 5.0"  
    }  
  }  
}  
  
provider "aws" {  
  region = "eu-west-1"  
}
```

variables.tf

```
variable "username" {  
  description = "Votre prenom - permet de differencier les ressources entre participants"  
  type        = string  
}
```

``main.tf``

```
resource "aws_security_group" "lab3_sg" {  
  name          = "lab3-sg-${var.username}"  
  description   = "Security group Lab 3 - ${var.username}"  
  
  tags = {  
    Name     = "lab3-sg-${var.username}"  
    Lab      = "lab3"  
    Username = var.username  
  }  
}  
  
resource "aws_instance" "lab3_instance" {  
  ami              = "ami-0694d931cee176e7d"  
  instance_type    = "t2.micro"  
  vpc_security_group_ids = [aws_security_group.lab3_sg.id]  
  
  tags = {  
    Name     = "lab3-instance-${var.username}"  
    Lab      = "lab3"  
    Username = var.username  
  }  
}
```

``outputs.tf``

```
output "instance_id" {  
  description = "ID de l'instance EC2"  
  value       = aws_instance.lab3_instance.id  
}
```

Déployez l'infrastructure :

```
terraform init  
terraform apply -var="username=<votre-prenom>"
```

Tapez **yes** pour confirmer.

■ Étape 2 — Lister les ressources du State

```
terraform state list
```

Résultat attendu :

```
aws_instance.lab3_instance
aws_security_group.lab3_sg
```

■ Étape 3 — Inspecter une ressource

```
# Détails complets de l'instance
terraform state show aws_instance.lab3_instance
```

```
# Détails du Security Group
terraform state show aws_security_group.lab3_sg
```

■ *Le state contient **tous les attributs** retournés par AWS : ID, ARN, IP, subnets, etc. C'est la source de vérité de Terraform sur votre infrastructure.*

■ Étape 4 — Inspecter le fichier tfstate brut

```
# Voir l'ID de l'instance dans le state
cat terraform.tfstate | jq '.resources[] | select(.type=="aws_instance") | .instances[0].attributes.id'
```

```
# Voir tous les types de ressources dans le state
cat terraform.tfstate | jq '.resources[].type'
```

■ Étape 5 — Renommer une ressource dans le State

Renommez `lab3_instance` en `web_server` dans le state **sans recréer l'instance** :

```
terraform state mv aws_instance.lab3_instance aws_instance.web_server
```

Vérifiez :

```
terraform state list
```

Résultat attendu :

```
aws_instance.web_server
aws_security_group.lab3_sg
```

■ Le code `main.tf` est maintenant désynchronisé avec le state. Mettez à jour `main.tf` :

```
sed -i 's/lab3_instance/web_server/g' main.tf
sed -i 's/lab3_instance/web_server/g' outputs.tf
```

Vérifiez qu'il n'y a plus de diff :

```
terraform plan -var="username=<votre-prenom>"
```

Résultat attendu : No changes. Infrastructure is up-to-date.

■ Étape 6 — Desired State vs Current State

Modifiez le type d'instance dans `main.tf` pour simuler un changement de desired state :

```
resource "aws_instance" "web_server" {
  ...
  instance_type = "t2.small" # ← changement
  ...
}
```

```
terraform plan -var="username=<votre-prenom>"
```

■ Terraform détecte la différence entre le **desired state** (`t2.small` dans le code) et le **current state** (`t2.micro` dans AWS) et propose une modification.

Annulez le changement avant de continuer (remettez `t2.micro`).

■ Étape 7 — Refresh du State

Ajoutez manuellement un tag sur l'instance depuis la CLI AWS pour simuler une modification hors Terraform :

```
INSTANCE_ID=$(terraform output -raw instance_id)

aws ec2 create-tags \
  --resources $INSTANCE_ID \
  --tags Key=ModifiedOutside,Value=true
```

Rafraîchissez le state pour capturer ce changement :

```
terraform refresh -var="username=<votre-prenom>"
```

Vérifiez que le state a capturé le nouveau tag :

```
terraform state show aws_instance.web_server | grep ModifiedOutside
```

■ Étape 8 — Nettoyage

```
terraform destroy -var="username=<votre-prenom>"
```

Tapez **yes** pour confirmer.

■ Validation du Lab

#	Critère	Validé
1	`terraform state list` liste les 2 ressources	■
2	`terraform state show` affiche les attributs AWS complets	■
3	`terraform state mv` renomme sans recréer la ressource	■
4	`terraform plan` retourne "No changes" après le renommage	■
5	`terraform plan` détecte le changement de `instance_type`	■
6	`terraform refresh` capture le tag ajouté manuellement	■
7	`terraform destroy` supprime toutes les ressources	■

■■ Résolution des problèmes courants

Problème	Cause	Solution
`state mv` échoue	Mauvais nom de ressource	Vérifier avec `terraform state list`
`No such resource` après `sed`	Fichier mal mis à jour	Vérifier avec `cat main.tf`
`terraform refresh` ne trouve pas les tags	Tags non propagés	Attendre 10s et réessayer

■ **Fin du Lab 3 — Vous maîtrisez la gestion du State Terraform !**

*Le Lab 4 introduit les **variables**, **locals**, **count**, **for_each**, **dynamic blocks** et les **modules**.*